Wide-Column Stores

Cassandra table design for a chat app

Watch the 3-min video

In one line: a wide-column store keeps rows in partitions chosen by a partition key and sorted by a clustering key. Design one table per query, so each read hits one partition, and bucket the partition key so no partition grows without limit.

1 The problem: billions of chat messages

A chat app stores **billions of messages**. Every time you open a channel, it must show the **latest 50 messages**, instantly, while millions of new messages are written every minute. Discord solved exactly this with a wide-column store.

2 Partitions of sorted rows

A wide-column store, like Cassandra, groups rows into **partitions**.

- The **partition key** is hashed to decide which nodes store the partition (and its replicas).
- Inside a partition, rows are **sorted by the clustering key**, so reading a slice in order is fast.
- One partition can hold thousands of rows, which is why they are called **wide rows**.

Do not confuse this with **columnar analytics databases** (Redshift, BigQuery, ClickHouse) that store each column separately for scans. Wide-column is about partitions of sorted rows.

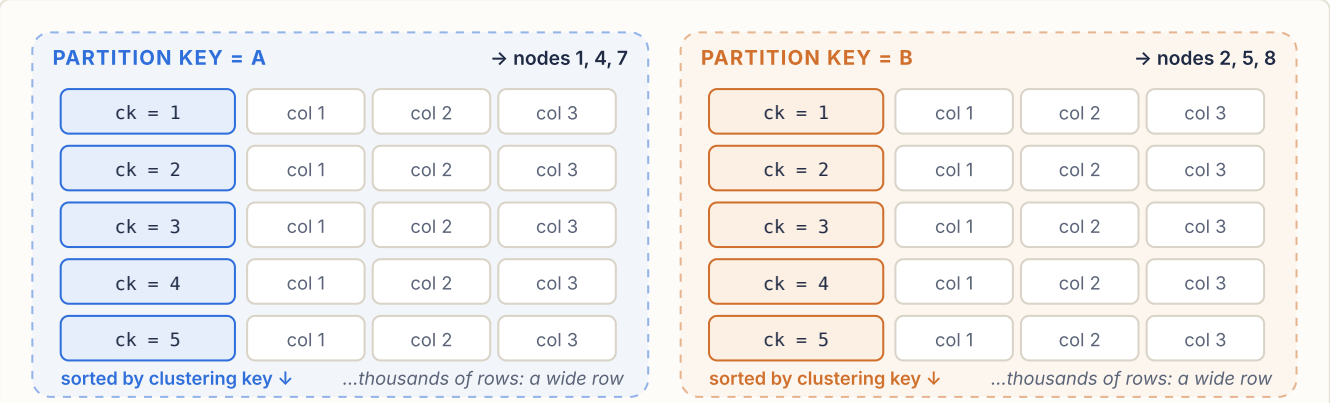


Figure 1. The partition key picks the replicas. The clustering key orders rows inside the partition, on disk.

3 Design from the queries, not the entities

There are **no joins** and filtering is limited, so every query should hit **one partition by its key**. You list the queries first, then create **one table per query**, and **denormalize**: the same data is written into several tables. Writes are cheap (an append to a log and a memtable), so duplication is fine.

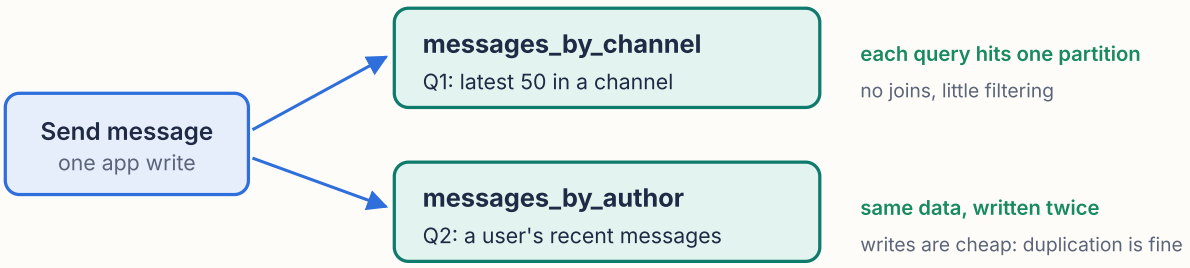


Figure 2. Two queries, two tables. The app (or a batch) writes each message to both; each read is a single-partition lookup.

4 The messages table

The **partition key** is the channel ID plus a **time bucket**, say 10 days. The **clustering key** is the message ID: a time-sortable **Snowflake ID** (a 64-bit number that starts with a timestamp), stored **newest first**. To load a channel, read one partition and take the first 50 rows. It is a single, sequential read.

```
CREATE TABLE messages (
  channel_id bigint,
  bucket     int,      -- message time in days / 10
  message_id bigint,   -- Snowflake ID: sorts by time
  author_id  bigint,
  content    text,
  PRIMARY KEY ((channel_id, bucket), message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);

SELECT * FROM messages
 WHERE channel_id = 42 AND bucket = 2048
 LIMIT 50;
```

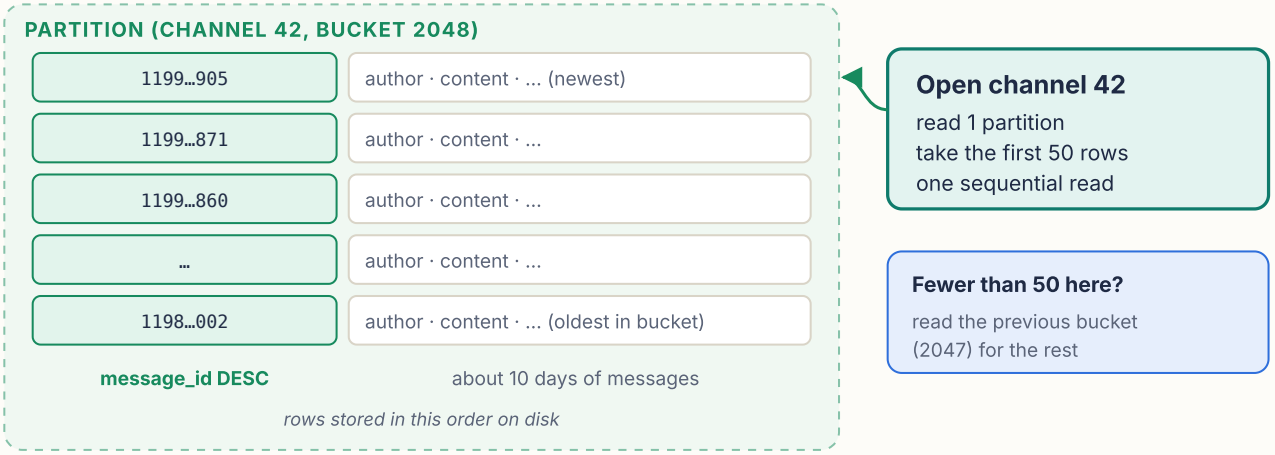


Figure 3. The latest messages sit at the top of the newest bucket. If a quiet channel has fewer than 50 there, the app also reads the bucket before.

5 Why the bucket: keep partitions bounded

Without a bucket, a busy channel's partition **grows forever**. A partition lives on one set of replicas, so huge partitions create **hot spots**, slow reads, and painful **compaction and repair**. A common guideline is to keep partitions **well under 100 MB**. Bucketing caps the size.

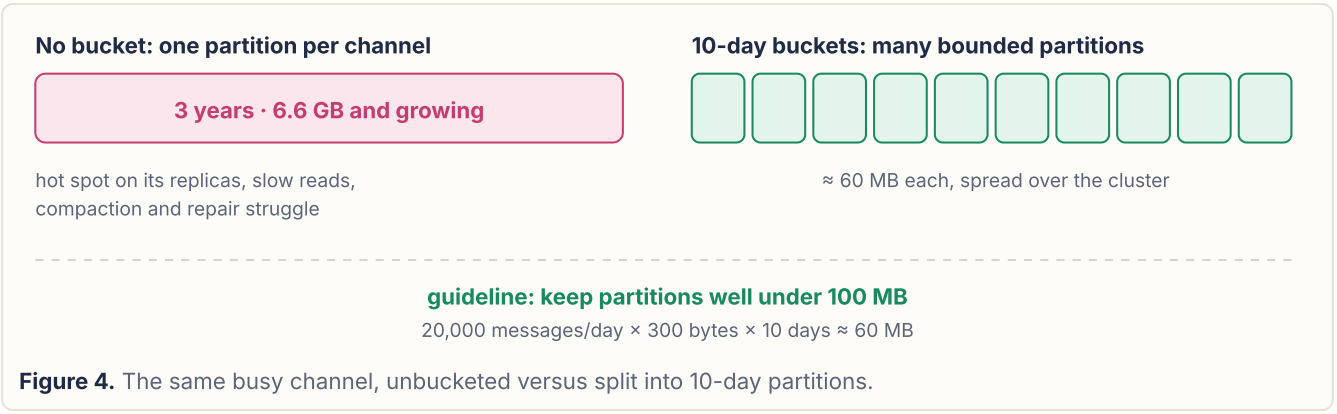


Figure 4. The same busy channel, unbucketed versus split into 10-day partitions.

WORKED EXAMPLE: CHOOSING THE BUCKET SIZE

- A busy channel gets **20,000 messages a day**, about **300 bytes** each: 6 MB a day.
- No bucket: after 3 years, 20,000 × 1,095 days × 300 B ≈ **6.6 GB** in one partition.
- 10-day bucket: 6 MB × 10 ≈ **60 MB** per partition. Under the guideline.
- If one channel were 10× busier, a 1-day bucket would keep it near 60 MB. Pick the bucket from your busiest partitions.

6 Deletes create tombstones

Cassandra stores data in an **LSM tree**: files are written once and never edited. So a delete writes a **marker called a tombstone**. The data is only purged during **compaction**, after a grace period (`gc_grace_seconds`, **10 days** by default), so replicas that missed the delete cannot bring the data back. Queries that scan many tombstones become slow, and Cassandra **fails a query after 100,000 of them** (it warns at 1,000). So avoid **queue-like patterns** that insert and delete constantly.

7 The family, and when it fits

- **Cassandra:** leaderless (any replica takes writes), with **tunable consistency** per query and the SQL-like **CQL** language.
- **ScyllaDB:** a compatible rewrite in C++. Discord moved its messages there in **2023**.
- **Google Bigtable** and its open-source cousin **HBase:** rows sorted by a single row key.

They all shine for **huge, write-heavy data with known queries** (messages, events, activity feeds, IoT readings) and struggle with **ad hoc queries and joins**.

COMMON MISTAKES

- Designing tables from entities, then finding a query needs `ALLOW FILTERING` across the cluster.
- Partitioning by channel alone, so busy channels grow into multi-GB partitions.
- Using a table as a work queue (insert, then delete): tombstones pile up and reads time out.
- Using random UUIDs as the clustering key, losing time order.

8 Key terms

Partition key: hashed to pick the nodes for a partition

Clustering key: sort order of rows inside a partition

Wide row: a partition with many rows

Denormalize: write the same data into each query's table

Bucket: a time slice added to the partition key

Snowflake ID: 64-bit ID that sorts by time

Tombstone: a delete marker, purged in compaction

gc_grace_seconds: wait before purging tombstones (10 days)

CQL: Cassandra Query Language

9 Interview questions

Q1. How do you design a Cassandra table for a chat app? BASIC

Start from the query. PRIMARY KEY ((channel_id, bucket), message_id) with CLUSTERING ORDER BY (message_id DESC), where message_id is a time-sortable Snowflake. Loading a channel reads the newest bucket's first 50 rows from one partition.

Q2. Partition key vs clustering key? BASIC

The partition key decides which nodes hold the data; every efficient query must give it. The clustering key orders rows within the partition, so range reads and “latest N” are a sequential slice.

Q3. Why avoid large partitions? BASIC

A partition lives on one replica set, so a huge one concentrates load on a few nodes, slows reads, makes compaction and repair painful, and can eventually fail. Keep them well under 100 MB, using a bucket.

Q4. You also need “all messages by user 7, newest first”. What changes? SCENARIO

Add a second table, messages_by_author, with PRIMARY KEY ((author_id, bucket), message_id) DESC, and write every message to both tables. Do not filter the channel table by author.

Q5. Pick a bucket size for channels with up to 200,000 messages a day at 300 bytes. SCENARIO

That is 60 MB a day, so a 1-day bucket gives about 60 MB per partition. Quiet channels then span many buckets, so read back bucket by bucket until you have 50 messages.

Q6. A team uses a Cassandra table as a job queue. Reads are getting slow. Why? SCENARIO

Each processed job is a delete, which writes a tombstone. Reading the head of the queue scans thousands of tombstones (kept for 10 days), and at 100,000 the query fails. Use Kafka, SQS or Redis for queues.

Q7. Is a wide-column store the same as a columnar database? TRICKY

No. Cassandra stores rows grouped in partitions for fast key lookups and writes. Columnar warehouses like BigQuery store each column separately to scan and aggregate billions of rows.

Q8. Why can't you just delete the data immediately instead of writing tombstones? TRICKY

SSTables are immutable, and a replica may have missed the delete. If the data vanished with no marker, repair would copy the old value back from that replica. The tombstone wins until every replica has seen it.

Next: SD-E-017 · Graph Databases Explained backendroadmap.com